

Signals Deep Dive

フロントエンド・PHPカンファレンス北海道 2026

nishitaku

自己紹介

- 西濃 拓郎（にしの たくろう） / @nishitaku
- フリーランス
- フロントエンドエンジニア（Angular）
- 奈良 > 神戸 > 川崎 > 名古屋 > 岐阜



Signals 知ってる人～ 🙌

これから話すこと

- Signals の本質は依存関係の追跡にある
- Signals がどうやって依存関係の追跡を実現しているか

Signals とは

- 状態と**依存関係**を効率的に扱うリアクティブモデル
- 各種フレームワーク、ライブラリで採用
- TC39 proposal-signals (Stage 1)

Signalsは何ができるのか？

例：カウンターと偶奇判定

```
let counter = 0;
const setCounter = (value) => {
  counter = value;
  renderParity();
}

const parity = () => (counter % 2) == 0 ? "even" : "odd";
const renderParity = () => element.innerText = parity();

setInterval(() => setCounter(counter + 1), 1000);
```

依存関係を知っておく必要がある

```
let counter = 0;
const setCounter = (value) => {
  counter = value;
  renderParity();
  renderColor(); // colorも変更しないとだめ
}

const parity = () => (counter % 2) == 0 ? "even" : "odd";
const renderParity = () => element.innerText = parity();

const color = () => (counter % 2) == 0 ? "blue" : "red";
const renderColor = () => counterEl.innerText = color();

setInterval(() => setCounter(counter + 1), 1000);
```

Signals の場合

- **State**: 手動で設定される値
- **Computed**: Stateに依存して計算される値
- **Effect**: StateやComputedに依存して実行されるコールバック

```
const counter = new Signal.State(0);

const parity = new Signal.Computed(() => (counter() % 2) == 0 ? "even" : "odd");

effect(() => {
  element.innerText = parity.get();
});

setInterval(() => counter.set(counter.get() + 1), 1000);
```


Signalsは何ができるのか？

例：宣言的UI

```
const name = "太郎";  
const age = 20;  
  
const validatedName = () => `${f(name)}`;  
const isEven = () => `${age} % 2 === 0 ? '偶数' : '奇数'`;
```

```
<p>名前は{{ validatedName }}です</p>  
<p>年齢は{{ isEven }}です。</p>
```

依存関係がわからない場合

```
const name = "太郎";  
const age = 21; // 20から変更された場合  
  
const validatedName = () => `${f(name)}`;  
const isEven = () => `${age} % 2 === 0 ? '偶数' : '奇数'`; // 再計算したい
```

```
<p>名前は{{ validatedName }}です</p>  
<p>年齢は{{ isEven }}です。</p>
```

- `isEven` だけではなく `validateName` も再計算する必要がある

Signals の場合

```
const name = signal("太郎");  
const age = signal(21); // 変更された場合  
  
const validatedName = computed(() => `${f(name)}`);  
const isEven = computed(() => `${age} % 2 === 0 ? '偶数' : '奇数'`);
```

```
<p>名前は{{ validatedName() }}です</p>  
<p>年齢は{{ isEven() }}です。</p>
```

- `isEven` だけ再計算される(自動依存関係トラッキング)
- 呼ばれてる場合だけ計算される(遅延評価)
- 同じ値なら再評価しない(メモ化)

Push型 or Pull型 ?

Push型：状態変更時に、依存先へ即座に通知・再計算する方式

例：Pub/Sub、WebSocket、プッシュ通知

- 常に最新状態
- 無駄な再計算が発生しやすい

Pull型：値が必要になった時に取得して計算する方式

例：ポーリング、関数呼び出し

- 必要なときに取得して計算
- 変更されていなくても取得してしまう

Signals は Push-Pull ハイブリッド方式

- Computedの評価は**Pull型**
- Stateの変更は即座に通知される**Push型**

**Signals はどうやって依存関係を解決
しているのか？**

例:カウンターと偶奇判定

```
const counter = signal(0);

const parity = computed(() => (counter() % 2) == 0 ? "even" : "odd");

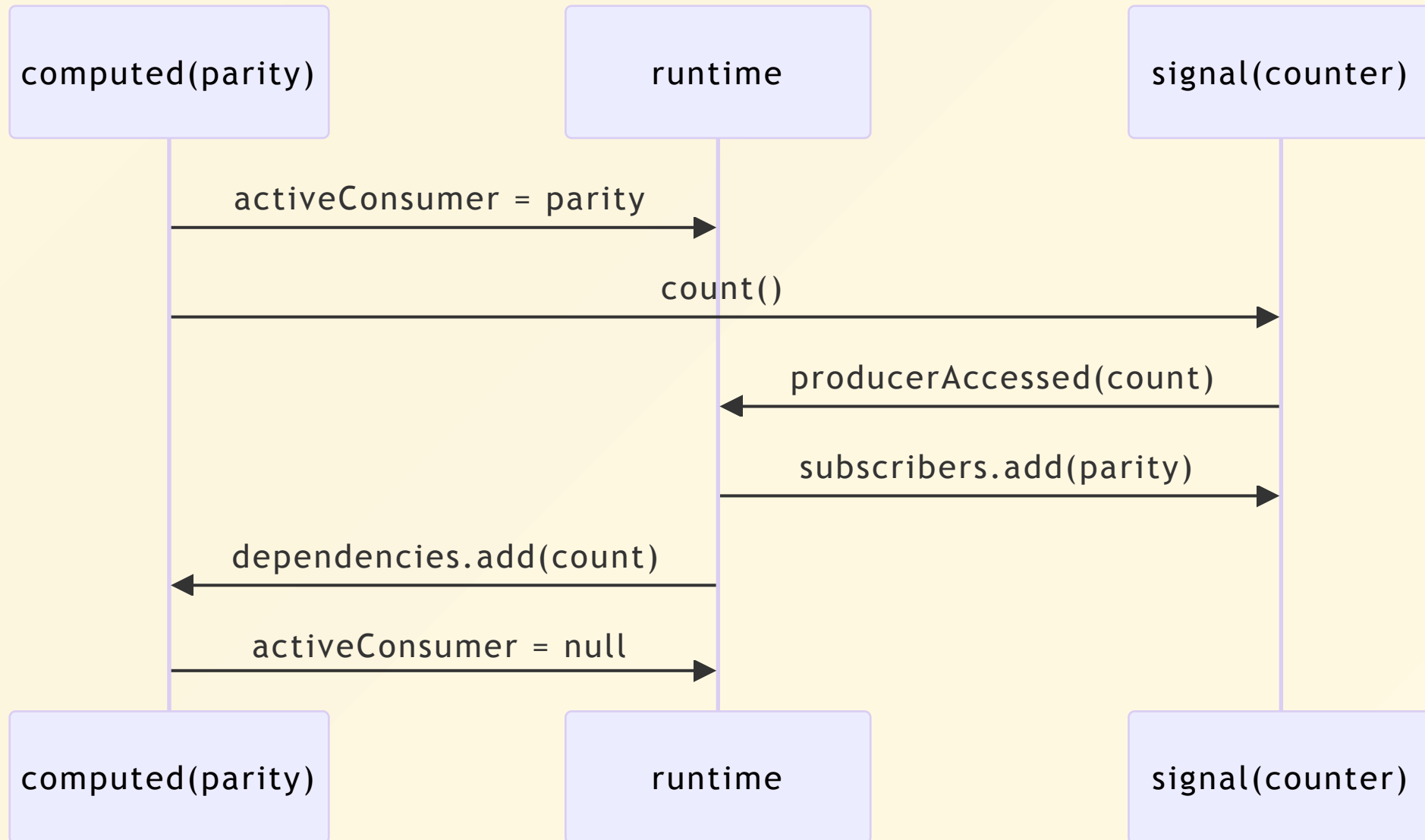
effect(() => {
  element.innerText = parity();
});
```

1. 依存関係の登録

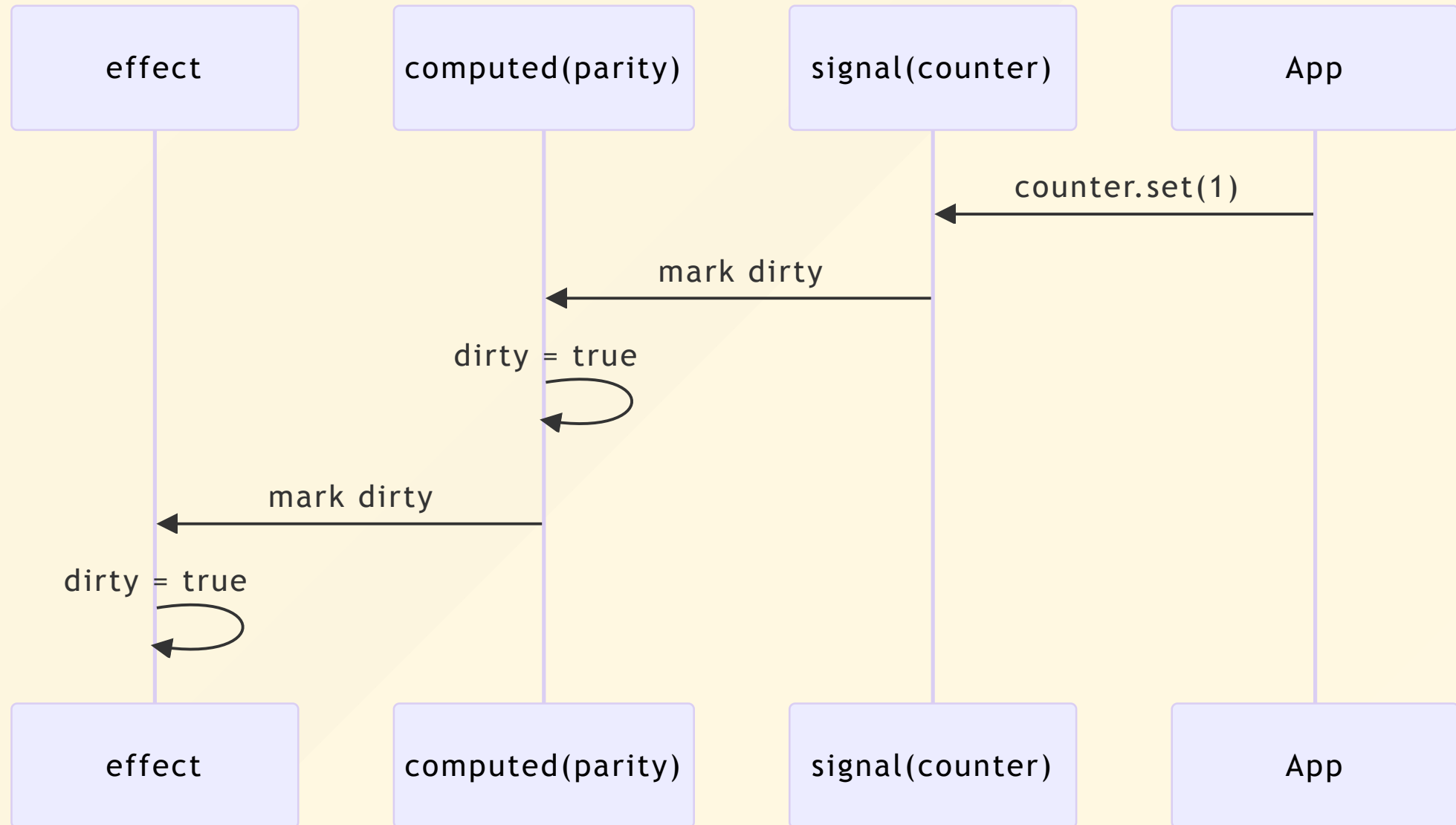
2. Push

3. Pull

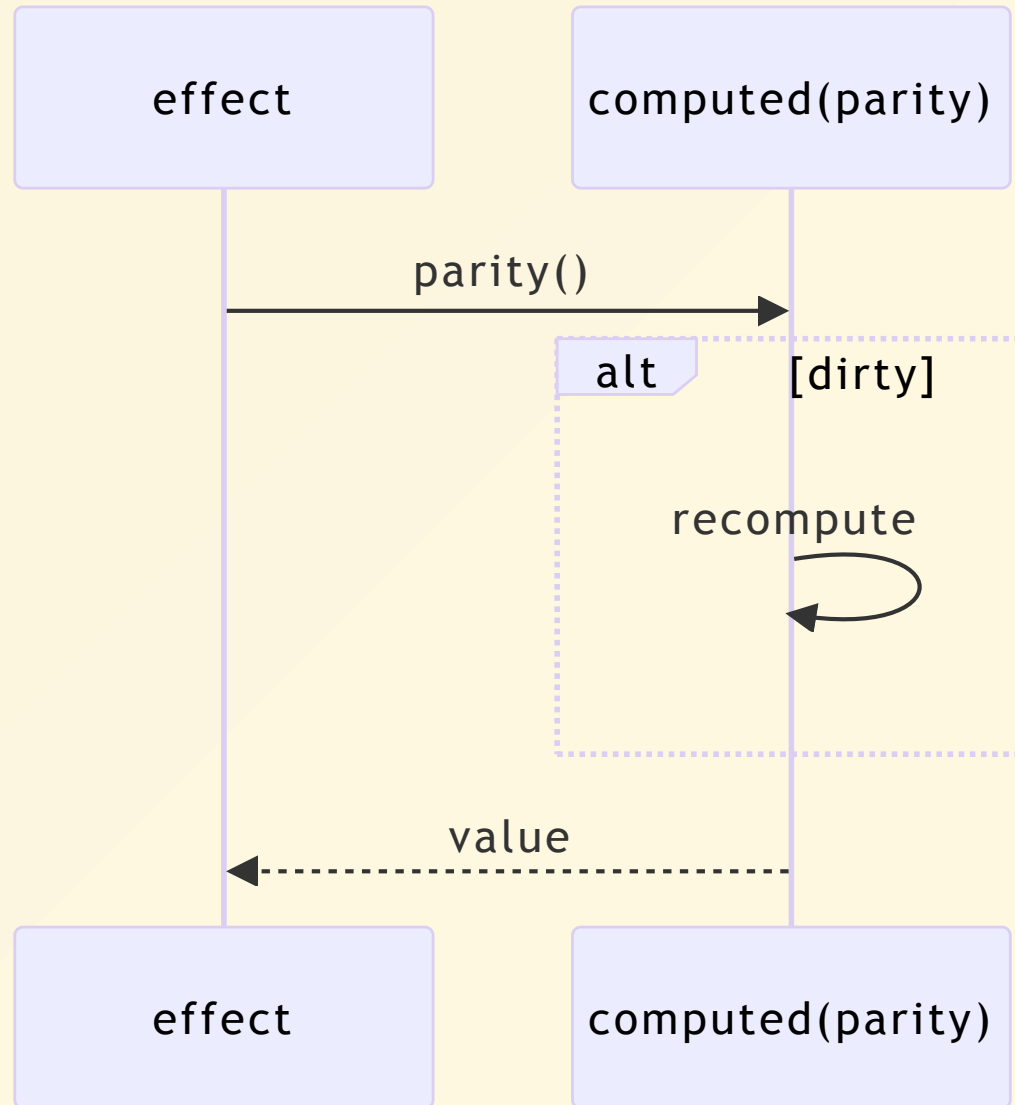
依存関係の登録



Push



Pull



まとめ

- Signalsは dependency graph を構築する
- dependency tracking によって依存関係を収集する
- Push/Pull Hybrid により効率的に更新する

ご清聴ありがとうございました

nishitaku

Runtime Dependency Tracking

- ビルド時ではなく、実行時に依存を解決している

```
const value = computed(() => {  
  if (flag()) {  
    return count()  
  }  
  
  return another()  
})
```

私とSignalsとの出会い

- Angular と zone.js
- v16 で 導入された Signals によって環境が激変
- Signals の魔法に感動して、仕組みが知りたくなった

signals のデメリット

グリッチフリー vs lossy

Q: 「グリッチフリー（不整合のない）」実行とはどういう意味ですか？

A: 初期のプッシュ型リアクティブモデルでは、State が変更されるたびに Computed が即座に走り、UI を更新しようとしていました。しかし、次のフレームまでに複数の変更がある場合、中間の不正確な値が表示される「グリッチ」が発生することがありました。Signal はプル型を採用することで、フレームワークが UI を描画するタイミングで必要な更新だけを取りに行くため、無駄な計算や DOM 操作、そしてその不整合を避けることができます。

Signals は observer pattern ?

- 広義のobserver pattern を含んでいる
- しかし本質は dependency graph
- Signalsは「読むことで依存登録」が重要
- computed による派生状態を扱える
- Push/Pull Hybrid によって lazy に再計算する

signal-polyfillを触ってみて

- 思ったよりシンプル
- 魔法ではない
- dependency graph の伝播アルゴリズム